資工碩二 611321246 黃柏瑞

一. 專案摘要

本專案旨在於 RISC-V 架構下，實作一個具備搶佔式多工能力的微核心，並重點深化行程間通訊機制的穩定性與功能性。

不同於基礎的信號量 (Semaphore) 實作，本專案完成了 RTOS 的通訊機制：

互斥鎖 (Mutex)：支援遞迴上鎖 (Recursive Locking) 與擁有權檢查 (Ownership)。

訊息佇列 (Message Queue)：基於環形緩衝區 (Ring Buffer) 的資料傳輸，支援動態記憶體配置。

事件標誌組 (Event Group)：支援複雜邏輯等待 (AND/OR)、廣播喚醒，並解決了虛假喚醒 (Spurious Wakeup) 問題。

二. 核心實作內容

A. 互斥鎖 (Mutex)

從傳統的 Semaphore 機制進化，實作了具備 擁有權 (Ownership) 與 遞迴 (Recursive) 特性的 Mutex。

資料結構設計：在 MutexCB 中新增 owner (紀錄持有者) 與 lockCount (遞迴計數)。

遞迴鎖 (Recursive Locking)：允許同一個 Task 多次獲取同一把鎖，避免自我死鎖 (Self-Deadlock)。

安全性機制：

擁有權檢查：在 release 時檢查是否為鎖的持有者，防止非法解鎖。

Ghost Wakeup 防禦：在 take 醒來後，再次確認鎖的狀態與擁有權，防止因超時或其他因素導致的 Lock Stealing 或狀態不一致。

系統呼叫整合：將 Kernel 端的 mut_take/release 封裝為 System Call，讓 User Mode 程式能透過 ecall 安全地存取 Kernel 資源。

B. 訊息佇列 (Message Queue)

實作了生產者-消費者 (Producer-Consumer) 模型：

Circular Buffer：利用 head 與 tail 指標管理佇列，有效利用記憶體空間。

資料隔離：使用 memcpy 進行資料搬運。User 傳入的是資料副本，而非直接操作 Kernel 內部的指標，確保 Kernel 記憶體的安全性。

Two-way Blocking：

Full：生產者寫入時若佇列已滿，進入 sendWaitList 等待。

Empty：消費者讀取時若佇列為空，進入 recvWaitList 等待。

C. 事件標誌 (Event Group)

這是本專案邏輯最複雜的部分，用於多個 Task 間的同步：

邏輯等待：支援 EVENT_WAIT_AND (所有條件滿足) 與 EVENT_WAIT_OR (任一條件滿足) 兩種模式。

自動清除 (Auto Clear)：支援在條件滿足並喚醒 Task 後，原子性地清除旗標。

廣播機制：當事件發生時，Kernel 會掃描整條等待清單，喚醒所有符合條件的 Task，而非只喚醒第一個。

三. 技術挑戰與解決方案

挑戰：在實作 Mutex 時，發現 Task 因 Timeout 醒來後沒有從等待清單移除，導致系統崩潰。

解決：在 mut_take 的超時處理路徑中，加入 list_remove 並增加狀態檢查。

挑戰： Spurious Wakeup 的防禦

問題描述：
在多工環境下，當一個 Task 因事件滿足被喚醒，但在它真正獲得 CPU 執行權之前，旗標可能已被其他高優先權的 Task 搶先清除。導致該 Task 醒來時，條件已不再滿足。

```
task0, event_wait::[Kernel] Woke up! Mode=0x00000006, CLR_Flag=0x00000004
task0, event_wait::[Kernel] Clearing Flags... Before: 0x00000000
task0, event_wait::[Kernel] Cleared! After: 0x00000000
task1, user_task1::Task 1: Created! I will trigger events.
task1, user_task1::
```

解決方案：

實作 二次驗證機制。

在 event_wait 函式中，當 Task 從睡眠中醒來時，不直接回傳成功，而是再次檢查目前的 Flags 是否真的符合條件。

- 若符合：執行 Auto Clear 並回傳 E_OK。
- 若不符合：判定為虛假喚醒，回傳 E_RETRY 或錯誤，防止 Task 執行錯誤邏輯。

四. 測試驗證

本專案設計了多個 User Task 場景進行壓力測試：

1. 遞迴鎖測試：

   Task 0 連續呼叫兩次 mutex_take，驗證系統不會死鎖，並正確釋放兩次後其他 Task 才能獲取。

2. 拼圖遊戲 (Event Logic)：

   Task A 等待 Bit 0 & Bit 1。

   Task B 分兩次分別設定 Bit 0 和 Bit 1。

   驗證 Task A 只有在第二次設定後才會醒來，證明 AND 邏輯運作正確。

3. 佇列傳輸：

   Task A (Sender) 連續發送資料，Task B (Receiver) 接收並驗證數據順序 (FIFO)，確認資料無遺漏或錯亂

五. 結論與未來展望 (Conclusion)

本專案成功在 RISC-V 平台上實作了一個功能完備的 IPC 子系統。透過親手實作 Mutex 的擁有權管理與 Event 的廣播邏輯，深入理解了 RTOS 在處理 Race Condition 與 資源保護 上的設計哲學。

雖然本專案未包含硬體驅動程式，但目前的 IPC 架構已具備高度的可移植性。未來的開發規劃是將此核心移植至 STM32 真實硬體上，並結合 NVIC 中斷控制器與 DMA，開發更高效的周邊驅動。